

Navigation

ไต้หวันมีภูเขาสูงหนาแน่นมาก โดยมียอดเขามากกว่า 250 ยอด ที่สูงกว่า 3,000 เมตร A-Ming วางแผนที่จะสร้างเครือข่ายขนส่งทาง zipline ด้วยหุ่นยนต์ในย่าน Central Mountain Range ของไต้หวัน เพื่อขนส่งพัสดุระหว่างหมู่บ้านที่อยู่ห่างไกล

ภายใต้แผนการของ A-Ming แต่ละเครือข่ายจะประกอบไปด้วยโหนดจำนวน N โหนด, มีหมายเลขตั้งแต่ 0 ถึง $N - 1$.

ในบรรดาโหนดเหล่านี้ จะมีโหนดจำนวน $K = 6$ โหนด **พอดิ** ที่เป็นสถานีพลังงาน และโหนดที่เหลือจำนวน $N - K$ โหนดจะเป็นโหนดหมู่บ้าน โหนดเหล่านี้จะเชื่อมต่อกันด้วยเคเบิล **สองทิศทาง** จำนวน $N - 1$ เส้น ที่มีหมายเลขตั้งแต่ 0 ถึง $N - 2$ สำหรับแต่ละ i ($0 \leq i \leq N - 2$), เคเบิล i จะเชื่อมโหนด $U[i]$ และ $V[i]$ เคเบิลแต่ละเส้นจะเชื่อมโหนดสองโหนดที่แตกต่างกัน และแต่ละคู่ของโหนดจะเชื่อมต่อกันด้วยเคเบิลไม่เกินหนึ่งเส้น รับประกันว่าสามารถเคลื่อนย้ายระหว่างคู่ของโหนดใด ๆ ได้ด้วยชุดของเคเบิลเหล่านี้

ระยะทาง ระหว่างโหนด a และ b เขียนแทนด้วย $d(a, b)$ และมีนิยามดังต่อไปนี้

- ถ้า $a = b$ แล้ว $d(a, b) = 0$.
- ไม่เช่นนั้น $d(a, b)$ จะเท่ากับจำนวนของเคเบิลที่ต้องใช้ในการเคลื่อนย้ายจาก a ไปยัง b

เราจะกล่าวว่า **branching factor** ของเครือข่ายจะมีค่าน้อย δ ถ้าแต่ละโหนดในเครือข่ายนั้นติดกับเคเบิลจำนวน 1 เส้นพอดิ หรือติดกับเคเบิลอย่างน้อย δ เส้น A-Ming ทราบจำนวนเต็มบวก B ที่รับประกันว่า branching factor ของเครือข่ายมีค่าน้อย B

A-Ming ได้เตรียมเคเบิลไว้แตกต่างกัน 100 ประเภท มีหมายเลข $0, 1, \dots, 99$ เคเบิลแต่ละเส้นในเครือข่ายจะต้องเป็นหนึ่งในประเภทเหล่านี้

หุ่นยนต์จะวิ่งไปตามเคเบิล zipline เหล่านี้ เพื่อที่จะทำงานส่งพัสดุให้สำเร็จ A-Ming ต้องการจะติดตั้งโปรแกรมสำหรับหาเส้นทางเพื่อช่วยให้หุ่นยนต์เดินทางกลับไปยังโหนดสถานีพลังงานและชาร์จตนเองโดยอัตโนมัติ อย่างไรก็ตามหุ่นยนต์มีหน่วยความจำน้อยมาก ดังนั้นในการออกแบบโปรแกรมหาเส้นทาง จะต้องทำให้หุ่นยนต์หาเส้นทางโดยอาศัยจำนวนสถานีพลังงาน $K = 6$, ค่า branching factor B ที่รับประกัน, และประเภทของเคเบิลต่าง ๆ ที่ติดกับตำแหน่งปัจจุบันของหุ่นยนต์เท่านั้น

เมื่อหุ่นยนต์ต้องการหาเส้นทางโดยเริ่มที่โหนดหมู่บ้าน u ที่ติดกับเคเบิลอย่างน้อยสองเส้น หุ่นยนต์จะตรวจสอบเคเบิลทั้งหมดที่ติดกับ u ตามลำดับใด ๆ ที่ไม่สามารถระบุได้ (arbitrary order) จากนั้นโปรแกรมจะได้รับรายการที่ระบุประเภทของเคเบิลในลำดับดังกล่าว สำหรับแต่ละเคเบิล โปรแกรมจะต้องนับจำนวนสถานีพลังงานที่ถ้าเดินทางไปตามเคเบิลเส้นดังกล่าวจะลดระยะทางระหว่างหุ่นยนต์กับสถานีพลังงานเหล่านั้น

กล่าวโดยละเอียดดังนี้ ให้ $v_1, \dots, v_k$ ($k \geq 2$) เป็นโหนดที่ติดกับ u โดยตรงด้วยเคเบิล และ c_i ($1 \leq i \leq k$) เป็นประเภทของเคเบิลที่เชื่อมระหว่าง u และ v_i โปรแกรมจะได้รับค่า K , B , และรายการ $c_1, c_2, \dots, c_k$ สำหรับแต่ละ i

($1 \leq i \leq k$), โปรแกรมจะต้องระบุจำนวนสถานีพลังงาน p ที่ $d(v_i, p) < d(u, p)$

งานของคุณคือกำหนดแนวทางที่จะกำหนดประเภทของเคเบิลและออกแบบโปรแกรมหาเส้นทาง คะแนนของคุณจะขึ้นกับจำนวนประเภทของเคเบิลที่แตกต่างกันที่คุณใช้ (ดูรายละเอียดเพิ่มเติมได้จากส่วน Subtasks and Scoring)

Implementation details

คุณจะต้องเขียนสองฟังก์ชัน ฟังก์ชันหนึ่งจะกำหนดประเภทของเคเบิล อีกฟังก์ชันจะเป็นโปรแกรมหาเส้นทางของหุ่นยนต์

ฟังก์ชันที่คุณจะต้องเขียนเพื่อ **กำหนดประเภทของเคเบิล** คือ:

```
std::vector<int> construct_network(int N, int K, int B,
                                   std::vector<int> U, std::vector<int> V,
                                   std::vector<int> P)
```

- N : จำนวนโหนด.
- K : จำนวนสถานีพลังงาน.
- B : ค่า branching factor ขั้นต่ำของเครือข่ายที่รับประกัน.
- U, V : อาร์เรย์ความยาว $N - 1$ ที่ระบุการเชื่อมต่อของเคเบิล.
- P : อาร์เรย์ความยาว $K = 6$ ระบุนโหนดที่เป็นสถานีพลังงาน.
- ฟังก์ชันนี้จะถูกเรียก **ไม่เกิน 10 000 ครั้ง** ในแต่ละข้อมูลชุดทดสอบ.

ฟังก์ชันจะต้องคืนอาร์เรย์ T ที่มีความยาว $N - 1$:

- เคเบิลประเภท $T[i]$ จะถูกใช้ในการเชื่อมต่อโหนด $U[i]$ และ $V[i]$
- แต่ละค่า $T[i]$ จะต้องสอดคล้องกับเงื่อนไข $0 \leq T[i] < 100$.

ฟังก์ชันที่คุณจะต้องเขียนสำหรับ **โปรแกรมของหุ่นยนต์** คือ:

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K : จำนวนสถานีพลังงาน.
- B : ค่า branching factor ขั้นต่ำของเครือข่ายที่รับประกัน.
- C : รายการของประเภทของเคเบิลที่ติดกับโหนดหมู่บ้านบางโหนดที่ติดกับเคเบิลอย่างน้อย 2 เส้น ในลำดับที่ไม่สามารถระบุได้ (arbitrary order).
- รับประกันว่าความยาวของ C จะไม่น้อยกว่า B
- ฟังก์ชันจะถูกเรียกไม่เกิน 100 000 ครั้ง ต่อหนึ่งกรณีทดสอบ.
- ฟังก์ชัน `navigate` จะต้องไม่ใช่โครงสร้างของเครือข่ายตั้งต้น; และในกรณีนี้ ระบบตรวจอาจเรียงลำดับในการเรียกฟังก์ชัน `navigate` ใหม่โดยไม่จำเป็นต้องเหมือนกับลำดับในการเรียกให้ระบุประเภทของเคเบิลในกรณีทดสอบเดียวกัน

ฟังก์ชันจะคืนอาร์เรย์ D :

- ความยาวของอาร์เรย์ D จะต้องเท่ากับความยาวของอาร์เรย์ C .
- $D[i]$ จะต้องเป็นจำนวนสถานีพลังงานที่ถ้าเดินทางตามเคเบิลที่สอดคล้องกับ $C[i]$ จะลดระยะทางระหว่างหุ่นยนต์กับสถานีพลังงานนั้น

ในระบบตรวจ ฟังก์ชัน `construct_network` และฟังก์ชัน `navigate` จะถูกเรียกในโปรเซส **สองโปรเซสที่แยกจากกัน**

Constraints

- $K = 6$.
- $K < N \leq 100\,000$.
- ผลรวมของ N ของทุกการเรียกฟังก์ชัน `construct_network` จะไม่เกิน 100 000 ในแต่ละกรณีทดสอบ.
- $0 \leq U[i] < V[i] < N$ สำหรับ $0 \leq i \leq N - 2$.
- เป็นไปได้ที่จะเดินทางจากโหนดใด ๆ ไปยังโหนดอื่น ๆ โดยใช้ระบบเคเบิลที่ระบุ.
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- แต่ละอาร์เรย์ C คือการเรียงสับเปลี่ยนของรายการของประเภทเคเบิลที่ติดกับบางโหนดหมู่บ้านที่ติดกับเคเบิลอย่างน้อย 2 เส้น
- ความยาวรวมของอาร์เรย์ C ในทุกการเรียกฟังก์ชัน `navigate` จะไม่เกิน 200 000 ในแต่ละกรณีทดสอบ.

Subtasks and Scoring

Subtask	คะแนน	เงื่อนไขเพิ่มเติม
1	6	$B = 2, N = 7$.
2	14	$B = 2, U[i] = i, V[i] = i + 1$ สำหรับแต่ละ i ที่ $0 \leq i < N - 1$.
3	20	$B = 5$.
4	20	$B = 4$.
5	40	$B = 2$.

ในกรณีทดสอบใด ถ้าเงื่อนไขข้อใดข้อหนึ่งต่อไปนี้เป็นจริง คะแนนของคุณในกรณีทดสอบนั้นจะเป็น 0 (ซึ่งจะแสดงว่า Output isn't correct ใน CMS):

- ค่าที่คืนจากการเรียก `construct_network` ไม่ถูกต้อง.
- ค่าที่คืนจากการเรียก `navigate` ผิดพลาด.

ไม่เช่นนั้น ให้ S แทนจำนวนเต็มที่น้อยที่สุดที่มากกว่าจำนวนเต็มทั้งหมดในทุก ๆ อาร์เรย์ T ที่คืนมาจากการเรียก `construct_network` หรือกล่าวอีกทางหนึ่งว่า S เป็นจำนวนเต็มน้อยที่สุดในเครือข่ายที่สร้างมาทั้งหมด จะใช้เคเบิลประเภท $0, 1, \dots, S - 1$ เท่านั้น

คะแนนของแต่ละปัญหาย่อยของคุณจะขึ้นกับ S ดังตารางต่อไปนี้:

Condition	Subtask 1	Subtask 2	Subtask 3 and 4	Subtask 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2\log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

อนึ่ง ในปัญหาย่อย 1, 2 และ 5, คุณจะได้คะแนนเต็มถ้า $S \leq 5$ และในปัญหาย่อย 3 และ 4, คุณจะได้คะแนนเต็มถ้า $S \leq 4$

Example

พิจารณาสถานการณ์ที่ $N = 9$, $K = 6$, และ $B = 2$.

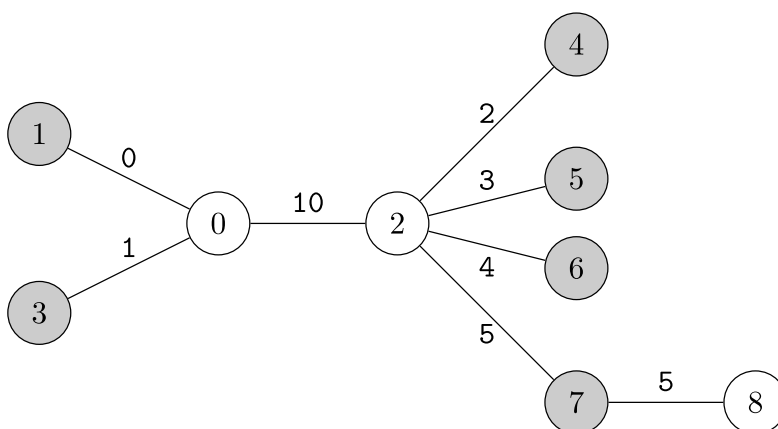
แผนการวางเครือข่ายระบุโดย $U = [0, 0, 0, 2, 2, 2, 2, 7]$, $V = [1, 2, 3, 4, 5, 6, 7, 8]$ สถานีพลังงานคือ $P = [1, 3, 4, 5, 6, 7]$ พิจารณาการเรียกในขั้นตอนแรกดังต่อไปนี้:

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 7],
                  [1, 2, 3, 4, 5, 6, 7, 8],
                  [1, 3, 4, 5, 6, 7])
```

สมมติว่า A-Ming สร้างเครือข่ายโดยคืนค่า:

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

เครือข่ายที่สร้างขึ้นแสดงดังรูปต่อไปนี้



หลังจากนั้น พิจารณาขั้นตอนที่สอง สมมติว่าหุ่นยนต์จะต้องหาเส้นทางที่โหนด 0 เนื่องจากโหนด 0 นั้นติดกับโหนด 1, 2 และ 3 ถ้าหุ่นยนต์อ่านประเภทเคเบิลในลำดับเดียวกันนี้ จะมีการเรียกฟังก์ชันดังด้านล่าง:

```
navigate(6, 2, [0, 10, 1])
```

จากโครงสร้างเครือข่าย:

- สถานีพลังงานแรก ซึ่งคือโหนด 1 มีระยะทางสั้นที่สุดใกล้กว่าเมื่อพิจารณาจากโหนดตนเอง (โหนด 1) นั่นคือ, $0 = d(1, 1) < 1 = d(0, 1) < 2 = d(2, 1) = d(3, 1)$.
- สถานีพลังงานที่สอง ซึ่งคือโหนด 3 มีระยะทางสั้นที่สุดใกล้กว่าเมื่อพิจารณาจากโหนดตนเอง (โหนด 3) นั่นคือ, $0 = d(3, 3) < 1 = d(0, 3) < 2 = d(1, 3) = d(2, 3)$.
- สำหรับสถานีพลังงานอื่น ๆ นั่นคือโหนด 4, 5, 6, และ 7, จะมีระยะทางสั้นที่สุดใกล้กับโหนด 2 มากกว่า นั่นคือ, $1 = d(2, u) < 2 = d(1, u) < 3 = d(0, u) = d(3, u)$ เมื่อ $u = 4, 5, 6$, หรือ 7.

ดังนั้น จำนวนของสถานีพลังงานที่มีระยะทางลดลงภายหลังจากการเคลื่อนที่ไปตามเคเบิล (0, 1), (0, 2), และ (0, 3) คือ 1, 4 และ 1, ตามลำดับ ดังนั้นฟังก์ชันจะต้องคืน:

```
[1, 4, 1]
```

สังเกตว่า `navigate` อาจจะถูกเรียกด้วยลำดับใน C ที่แตกต่างกันไป (จะเป็นการเรียงสับเปลี่ยน) ตัวอย่างเช่น เป็นไปได้ที่จะมีการเรียก `navigate(6, 2, [1, 0, 10])` ที่โหนด 0 ฟังก์ชันจะต้องคืนคำตอบเป็น `[1, 1, 4]` ให้สอดคล้องกับข้อมูลนำเข้า

สมมติว่าหุ่นยนต์จะต้องหาเส้นทางที่โหนด 2. จะมีการเรียกฟังก์ชันดังนี้:

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

ฟังก์ชันจะต้องคืน:

```
[2, 1, 1, 1, 1]
```

ในเครือข่ายนี้ ฟังก์ชัน `navigate` จะไม่ถูกเรียกที่โหนดอื่น เนื่องจาก:

- โหนด 1, 3, 4, 5, 6 และ 7 ล้วนเป็นสถานีพลังงาน และ
- โหนด 8 ติดกับเคเบิลเดียวเท่านั้น

ในกรณีทดสอบนี้ ประเภทของเคเบิลที่เราใช้คือ 0, 1, 2, 3, 4, 5, และ 10 ดังนั้น $S = 11$ จะถูกใช้เพื่อคำนวณคะแนนที่ได้รับ

นอกเหนือจากตัวอย่างนี้ ใน attachment package สำหรับโจทย์ข้อนี้ จะมีตัวอย่างข้อมูลนำเข้าและข้อมูลส่งออกอีกสองตัวอย่าง

Sample Grader

เกรดเดอร์ตัวอย่างจะเรียกทั้งฟังก์ชัน `construct_network` และ `navigate` ในโปรเซสเดียวกัน นอกจากนี้ เมื่อเกรดเดอร์ตัวอย่างเรียก `navigate` ประเภทของเคเบิลจะแสดงตามลำดับเดียวกับลำดับของเคเบิลที่ปรากฏในข้อมูลนำเข้า พฤติกรรมทั้งสองนี้ แตกต่างจากในเกรดเดอร์จริง

รูปแบบข้อมูลนำเข้า:

```
T
(scenario 0)
(scenario 1)
...
(scenario T-1)
```

แต่ละสถานการณ์จะอยู่ในรูปแบบดังนี้:

```
N
B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]
```

ในแต่ละสถานการณ์ เกรดเดอร์ตัวอย่างจะเรียก `navigate` จำนวน Q ครั้ง ครั้งที่ i จะเป็นการเรียกด้วยข้อมูลของโหนด $X[i-1]$ สังเกตว่า เกรดเดอร์ตัวอย่างรองรับการใส่ค่า K ที่แตกต่างจาก 6

รูปแบบข้อมูลส่งออก:

ถ้าอาร์เรย์ที่คืนจาก `construct_network` หรือ `navigate` ผิดพลาด เกรดเดอร์ตัวอย่างจะจบการทำงานและพิมพ์เหตุผล ไม่เช่นนั้น เกรดเดอร์ตัวอย่างจะแสดงผลอาร์เรย์ D แต่ละอันที่คืนจาก `navigate` แยกทีละบรรทัด

หลังการดำเนินการทุกสถานการณ์ เกรดเดอร์ตัวอย่างจะพิมพ์ข้อมูลหนึ่งบรรทัดทาง `standard error`:

```
S = S'
```

โดยที่ S' คือจำนวนเต็มที่น้อยที่สุดที่มากกว่าทุก ๆ จำนวนในอาร์เรย์ T ทั้งหมดที่คืนจากการเรียก `construct_network` ในกรณีทดสอบนี้